IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Entrenar bien: sobreajuste y regularización

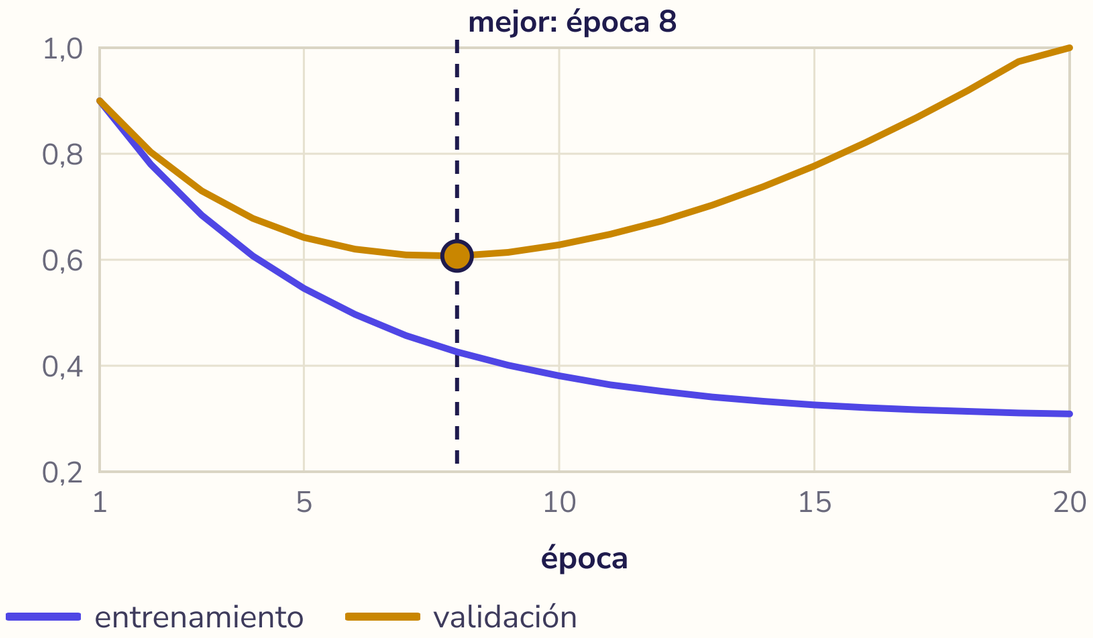
Teoría, práctica y, al final, material de consulta

Sprint 10 · S10_03 · teoría breve (25 min el 17/11), 55 min de práctica en clase (18/11) y material de consulta

LA IDEA · MEMORIZAR

Una red que memoriza en lugar de aprender

Pérdida por época (ejemplo inventado)



Muchos parámetros, pocos datos

La red acaba **aprendiéndose los ejemplos de memoria**, incluido el ruido.

Cómo se ve

El entrenamiento sigue bajando; la validación **toca fondo y vuelve a subir**.

Brecha de generalización

La distancia entre las dos curvas: **cuanto mayor, más sobreajuste**.

LAS HERRAMIENTAS · SECCIÓN 2

Cuatro formas de frenar la memorización

Técnica	Idea en una frase	En Keras
L2 (weight decay)	Castiga los pesos grandes: la red prefiere soluciones «suaves»	`kernel_regularizer=keras.regularizers.l2(0.01)`
Dropout	En cada paso apaga al azar un % de neuronas: no puede depender de una sola	`keras.layers.Dropout(0.3)`
Parada temprana	Para cuando la validación deja de mejorar	callback `EarlyStopping`
Batch normalization	Reescala las activaciones en cada lote: entrena más estable y regulariza algo	`keras.layers.BatchNormalization()`

El remedio más barato suele ser más datos o un modelo más pequeño; estas herramientas reducen la brecha.

CÓMO ACTÚAN · L2 Y DROPOUT

L2 y dropout, con números pequeños

L2 suma una multa a la pérdida

pérdida total = error + λ · Σ w² con λ = 0,01

Dropout 0,4 apaga neuronas al azar

Las que siguen se multiplican por **1 / (1 − 0,4) = 1,67**.

Pesos de una neurona	Σ w²	Multa (λ = 0,01)
3 · −0,5 · 2	13,25	0,1325
1 · −0,5 · 2	5,25	0,0525

	n1	n2	n3	n4	n5
activación	0,8	1,5	0,2	1,0	0,6
¿se queda?	sí	**no**	sí	sí	**no**
al entrenar	1,33	0	0,33	1,67	0

El peso grande (3) es el que más multa pone, así que la red lo reduce. Dropout solo actúa al entrenar: al predecir se usan todas las neuronas.

CALLBACKS · SECCIÓN 3

Parar a tiempo y quedarse con lo mejor

Ejemplo con **patience=3**: la pérdida de validación por época.

época	1	2	3	4	5	6	7
pérdida val.	0,70	0,62	0,58	**0,57**	0,59	0,60	0,61
épocas sin mejorar	0	0	0	0	1	2	3

A la 7.ª para, y restore_best_weights=True devuelve los pesos de la época 4.

EarlyStopping

Para si la validación no mejora en **patience** épocas seguidas.

ReduceLROnPlateau

Si se estanca, baja la tasa de aprendizaje (**factor=0.5**: a la mitad).

ModelCheckpoint

Guarda en disco el mejor modelo (**save_best_only=True**).

Gradientes que desaparecen o explotan

Al retropropagar se **multiplica un factor por capa**: la sigmoide da **0,25 como máximo**; un peso de 1,5, ×1,5.

Capas	Factor 0,25 por capa: se apaga	Factor 1,5 por capa: se dispara
2	0,0625	2,25
5	0,00098	7,59
10	0,00000095	57,67
Síntoma	La pérdida apenas baja y las primeras capas casi no cambian	La pérdida salta de golpe o aparece **nan**

Remedio	En Keras
ReLU (derivada 1 si la entrada es positiva)	**activation="relu"**
Inicialización pensada para ReLU	**kernel_initializer="he_normal"**
Normalizar cada lote	**keras.layers.BatchNormalization()**
Recortar el gradiente	**keras.optimizers.Adam(clipnorm=1.0)**

AFINAR · TASA Y LOTE

Tasa de aprendizaje y tamaño de lote

Si la tasa es…	La curva de validación…	Qué hacer
Muy alta	salta, o toca fondo enseguida y **sube**	Bajarla
Adecuada	baja con suavidad y se estabiliza	Dejarla
Muy baja	baja tan despacio que **no llega**	Subirla o dar más épocas

Tamaño de lote	Efecto
Pequeño	Más actualizaciones por época y más ruido; cada época **tarda más**
Grande	Cada época es **más rápida**, pero hay menos actualizaciones

No hay un valor «correcto» universal: se prueba en escala logarítmica (0,1 · 0,01 · 0,001…) y se mira la curva y el tiempo.

BUENAS PRÁCTICAS · GUARDAR Y COMPARAR

Guardar el modelo y compararlo con honestidad

Qué guarda **.keras**

La **arquitectura**, los **pesos** y la configuración del optimizador.

Qué falta

El **preprocesado** (escalado y one-hot): guárdalo también, por ejemplo con **joblib**.

Referencia honesta

La misma red entrenada **«a ciegas»** hasta el final: es la cifra que hay que **superar**.

Un modelo guardado sin su preprocesado no sirve: los datos nuevos hay que tratarlos exactamente igual.

PUENTE AL RETO · S10_04 (DÍA 2)

De las curvas al veredicto del reto

1 · Misma partición

64 % entrenamiento, 16 % validación (decisiones y curvas) y 20 % prueba, **una sola vez**.

2 · Varias semillas

El MLP se entrena con **3 semillas**: media y variación, no un número suelto. Las curvas se leen con **diagnostico(...)**.

3 · Diferencia con incertidumbre

Si el intervalo de la diferencia de AUC incluye el 0, **no** se puede afirmar que uno sea mejor.

Ejemplo inventado	Diferencia de AUC (red – boosting)	Intervalo al 95 %	¿Incluye el 0?
A	+0,004	[–0,010 ; +0,018]	sí → no hay diferencia clara
B	+0,031	[+0,012 ; +0,050]	no → hay diferencia

Lo marcado «en casa» (tasa, lote, guardar, mini-reto) no es necesario para el reto.

PRÁCTICA · CUADERNO S10_03

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar herramientas y datos (800 reservas) y definir las funciones	Clase
Ejercicio 1	Época óptima y brecha final de la red grande	Clase
Ejercicio 2	Entrenar con dropout y comparar las recetas	Clase
Ejercicio 3	Parada temprana y reducción de la tasa	Clase
Para leer	Efecto de la tasa de aprendizaje y del tamaño de lote	Casa
Ejercicio 4	Guardar y cargar el modelo	Casa
Mini-reto	Gana a la red entrenada a ciegas	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

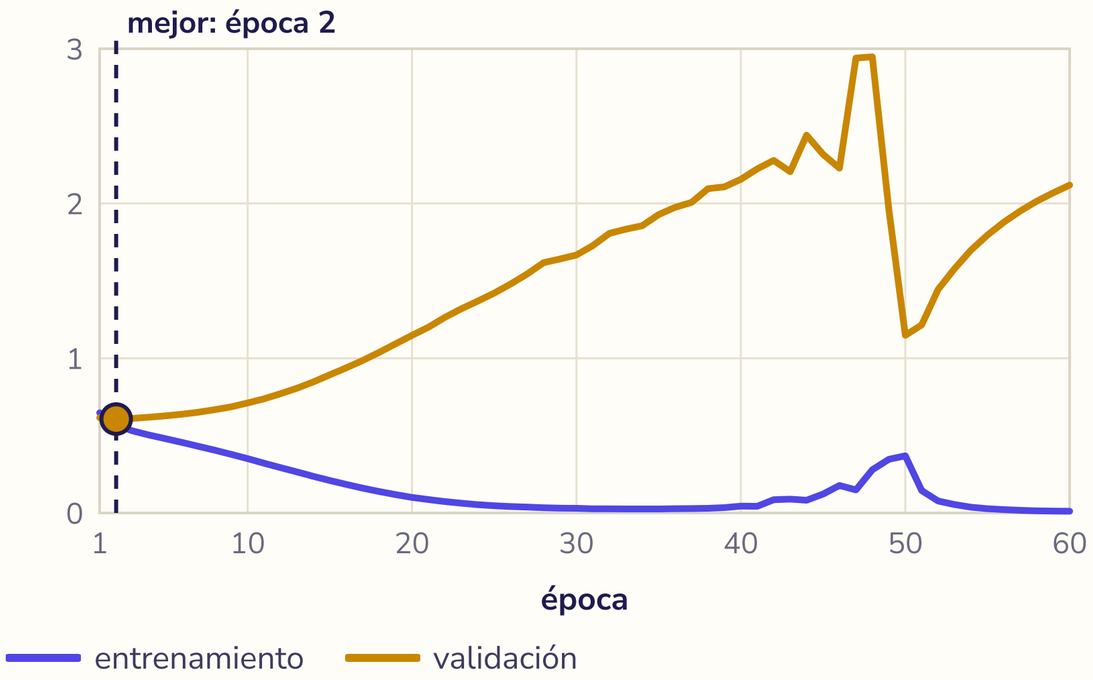
En el **mismo orden que el cuaderno**: ejercicios 1 a 4 y mini-reto, con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Época óptima y brecha final	12, 13 y 14
2	Dropout y comparación de recetas	15 y 16
3	Callbacks y parada temprana	17 y 18
Ejemplo	Tasa de aprendizaje y tamaño de lote	19
4	Guardar y cargar el modelo	20 y 21
Mini-reto	Ganar a la red a ciegas	22 y 23

CONSULTA · EJEMPLO · ENTRENAR CON POCOS DATOS

La red grande con 800 reservas memoriza

Pérdida por época · red 256-128 con solo 800 reservas



42 241

parámetros de la red, para **800** reservas

0,011 vs 2,12

pérdida de entrenamiento al final frente a la de validación (mínimo: 0,607)

Brecha de generalización

La distancia entre las dos curvas.

CONSULTA · EJERCICIO 1

¿En qué época empezó a memorizar?

```
perd_val = h_base.history["val_loss"]
perd_ent = h_base.history["loss"]
epoca_optima = int(np.argmin(perd_val)) + 1
brecha_final = perd_val[-1] - perd_ent[-1]
```

2

época de validación mínima, contando desde 1 (de 60)

2,11

brecha final = validación − entrenamiento

Época	1	2	5	10	20	40	60
Pérdida de entrenamiento	0,648	0,569	0,480	0,351	0,101	0,044	0,011
Pérdida de validación	0,616	**0,607**	0,628	0,711	1,148	2,156	2,119

Si la mejor época está muy por debajo del total, seguir entrenando solo empeoraba el modelo.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la época óptima y la brecha (ejercicio 1)

np.argmin(perd_val) + 1

argmin da la **posición** (desde 0); las épocas se cuentan desde 1, de ahí el **+ 1**.

perd_val[-1] - perd_ent[-1]

Validación **menos** entrenamiento. Al revés, la brecha saldría negativa y el **assert** se queja.

2 y 2,11

Mejor época: **2 de 60**. Brecha final: 2,119 – 0,011 ≈ **2,11**.

¿Qué concluyes?

Seguir entrenando **solo empeoraba** el modelo: había que parar mucho antes (y regularizar).

CONSULTA · EJERCICIO 2

Cinco recetas contra el sobreajuste

```
m_drop, h_drop = entrenar(A_peq, y_peq, epocas=60, dropout=0.4)
n_dropout = sum(isinstance(capa, keras.layers.Dropout) for capa in m_drop.layers)  # = 2
```

Receta (800 reservas, 60 épocas)	Mejor pérdida val.	Pérdida val. final	AUC val. final	AUC val. máximo
Sin regularizar	0,607	2,119	0,640	0,728
L2 = 0,01	0,696	1,008	0,667	0,731
Dropout 0,4	0,609	1,206	0,652	0,728
BatchNorm	0,645	1,509	0,672	0,701
L2 + dropout	0,743	0,936	0,683	0,731

Bajan la pérdida final (2,12 → 0,94–1,51), pero el AUC máximo apenas se mueve: no crean información.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer el dropout y las recetas (ejercicio 2)

dropout=0.4

Añade una capa **Dropout(0.4)** **tras cada capa oculta**: 2 en total (256 y 128). Apaga el 40 % de las neuronas al azar en cada paso.

isinstance(capa, Dropout)

Cuenta las capas de ese tipo del modelo: el **assert** espera **2**.

Con dropout 0,4

Pérdida de validación final ≈ **1,21** frente a ≈ **2,12** sin regularizar: sigue subiendo, pero mucho menos.

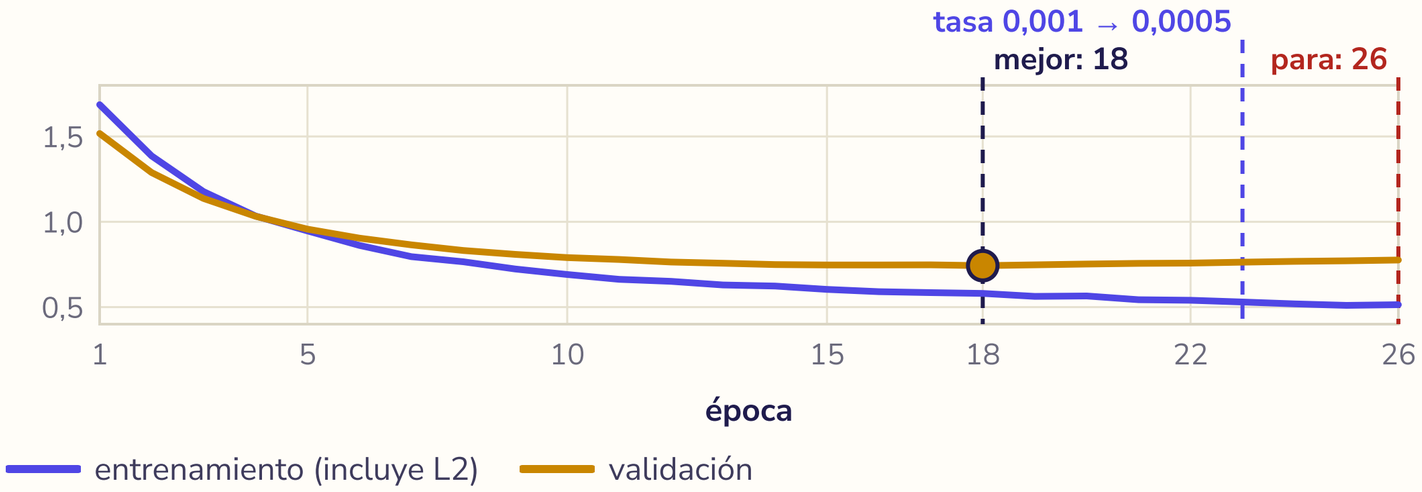
Los remedios no crean información

Bajan la pérdida final, pero el AUC máximo apenas se mueve. L2 suma su penalización a las dos pérdidas, por eso sale más alta.

CONSULTA · EJERCICIO 3

Callbacks: la red para sola en la época 26

Pérdida por época · EarlyStopping (paciencia 8) + ReduceLROnPlateau (paciencia 4)



26 / 100

épocas entrenadas, de 100 posibles (18 + paciencia 8)

0,743

mejor pérdida de validación, en la época 18

```
mis_callbacks = [EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True),
                 ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4)]
```

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer la parada temprana (ejercicio 3)

patience=8

Espera 8 épocas sin mejora: mejor época 18 + 8 = **26**. Por eso para en la 26 y no en la 18.

restore_best_weights=True

Devuelve los pesos de la **época 18**. Sin él, el modelo se quedaría con los de la última (la 26), algo peores.

callbacks=mis_callbacks

Hay que pasarlos a **entrenar**; si no, entrena las 100 épocas y el **assert** se queja.

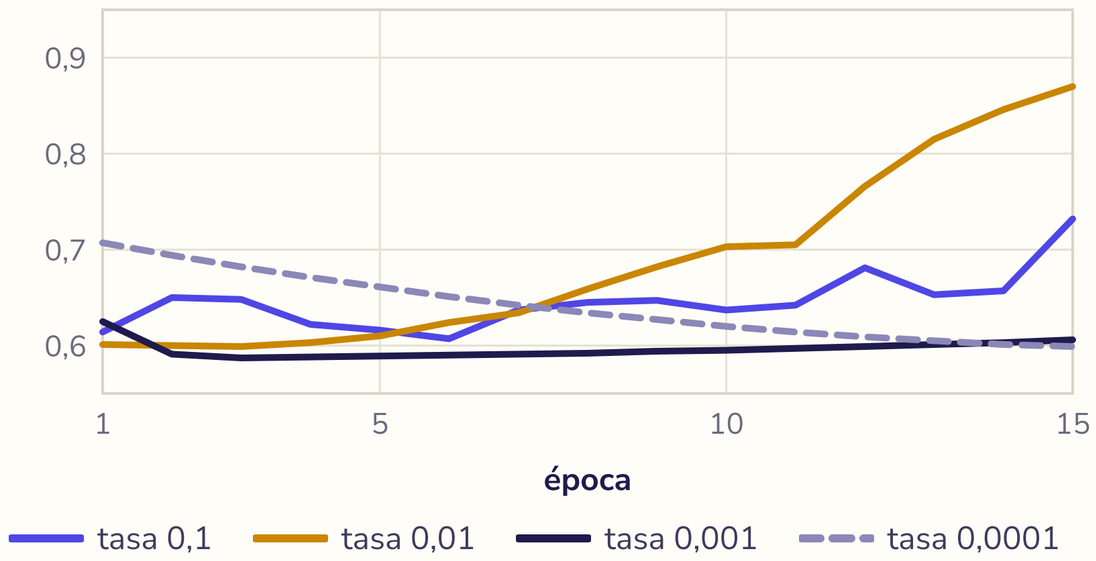
ReduceLROnPlateau

La tasa bajó **una vez**, de 0,001 a 0,0005 (en escala logarítmica: baja por factores, no por sumas).

CONSULTA · EJEMPLO · TASA Y TAMAÑO DE LOTE

Tasa de aprendizaje y tamaño de lote

Pérdida de validación por época · red 64-32, 3840 filas, lote 64



Prueba	AUC val. final	Segundos*
tasa 0,1	0,690	4,0
tasa 0,01	0,681	4,5
tasa 0,001	0,738	5,1
tasa 0,0001	0,737	4,0
lote 32	0,730	7,6
lote 128	0,742	5,1
lote 512	0,749	3,0

* Segundos en nuestra prueba, a título orientativo: en tu ordenador saldrán otros. 15 épocas por prueba.

CONSULTA · EJERCICIO 4

Guardar y cargar el modelo (ejercicio 4)

```
ruta_modelo = os.path.join(carpeta_tmp, "modelo_cancelaciones.keras")
m_es.save(ruta_modelo)
m_cargado = keras.models.load_model(ruta_modelo)
prob_cargado = m_cargado.predict(A_val, verbose=0)
np.allclose(prob_original, prob_cargado, atol=1e-6)    # True
```

525 KB

tamaño del archivo **.keras**

iguales

predicciones del modelo recargado

El archivo guarda arquitectura, pesos y optimizador, no el preprocesado: guárdalo también.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer guardar y cargar (ejercicio 4)

.keras guarda…

La **arquitectura**, los **pesos** y la configuración del optimizador: **525 KB** en el ejemplo.

np.allclose(prob_original, prob_cargado)

Comprueba que el modelo recargado da **las mismas probabilidades** que el original, sin reentrenar.

prob_cargado sale de **m_cargado**

Si se calculara con **m_es**, el **assert** pasaría sin probar nada: hay que predecir con el modelo **cargado**.

Falta el preprocesado

En producción hay que aplicar el mismo escalado y one-hot: guarda también el **ColumnTransformer** (con **joblib**).

CONSULTA · MINI-RETO

Gana a la red entrenada a ciegas (mini-reto)

```
_, h_ref = entrenar(A_ent, y_ent, epocas=40, lote=64, capas=(256, 128))
perdida_referencia = h_ref.history["val_loss"][-1]                  # 1.3336
m_final, h_final = entrenar(A_ent, y_ent, epocas=40, lote=64, capas=(64, 32), l2=0.003, dropout=0.3,
    callbacks=[keras.callbacks.EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)])
mejor_perdida_val = m_final.evaluate(A_val, y_val, verbose=0)[0]    # 0.6151
```

1,3336

referencia: red grande sin regularizar

0,6151

tu red: (64, 32) + L2 + dropout + parada

Menos pérdida que la referencia: una red más pequeña y las herramientas de hoy.

CONSULTA · ACLARACIONES · MINI-RETO

Leer la comparación con la referencia (mini-reto)

Referencia: 1,3336

Red grande **sin regularizar** entrenada 40 épocas: la mejor había sido 0,5915, pero al final queda en 1,3336.

La solución usa…

Red **(64, 32)** con L2 0,003, dropout 0,3 y **EarlyStopping**: pérdida **0,6151**.

Qué aporta cada pieza

Red pequeña: **menos capacidad** para memorizar. L2 y dropout: frenan el sobreajuste. **EarlyStopping**: devuelve el mejor modelo.

¿Hay que bajar de 0,5915?

No: basta con **bajar de 1,3336**. Con otra combinación podría incluso acercarse a la mejor época de la referencia.